5–6 장 예습과제

목차

5. 토큰화

단어 및 글자 토큰화

형태소 토큰화

형태소 어휘 사전 / KoNLPy / NLTK / spaCy

하위 단어 토큰화

바이트 페어 인코딩 / 센텐스피스 / 워드피스

6. 임베딩

원-핫 인코딩

언어 모델 / 자기회귀 언어 모델

N-gram

TF-IDF (단어 빈도 / 문서 빈도 / 역문서 빈도)

TF-IDF 모델 실습

5. 토큰화

- 자연어(Natural Language)는 자연 언어라고도 부르며, 인공적으로 만들어진 프로그래밍 언어와 다르게 사람들이 일상적으로 사용하는 언어
- 자연어 처리(Natural Language Processing, NLP)는 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술
- 자연어 처리는 음성·문장·문서 등 다양한 형태의 언어 데이터를 다룸
- 컴퓨터가 텍스트를 바로 이해할 수 있는 것은 아니므로 데이터를 일정한 처리 단위인 토큰(Token)으로 나누고, 이후 어휘 사전(Vocabulary)에 대응시켜 수치화하는 과정이 필요
- 모호성(Ambiguity): 같은 단어라도 문맥에 따라 여러 의미를 가질 수 있어 언어를 해석할 때 어려움 발생
- 가변성(Variability): 같은 의미를 전달하는 표현이 여러 개일 수 있고, 오탈자·줄임말·구어체 등 표현 방식이 다양함
- 구조화(Structure): 문장은 단어의 배열 및 문법적 관계에 따라 구성되며, 어순과 문맥에 따라 의미가 달라질 수 있음
- 토큰화(Tokenization)는 텍스트를 모델이 다루기 쉬운 단위로 나누는 과정
- 어떤 단위를 선택하는지에 따라 토큰의 개수, 어휘 사전의 크기, 미등록 단어(OOV, Out of Vocabulary)의 발생 여부가 달라짐

단어 및 글자 토큰화

1. 단어 토큰화(Word Tokenization)

- 단어 토큰화는 문장이나 텍스트를 의미 있는 단어의 경계에 따라 분리하는 방식
- 가장 간단한 접근은 공백(Whitespace) 분리이지만, 실제 텍스트에는 구두점과 문장부호가 들어 있으므로 단순 공백 분리만으로 원하는 결과를 얻기 어려움
- 영어 문장에서 공백과 구두점을 고려하면 단어를 분리할 수 있지만, 한국어에서는 조사와 어미가 결합하므로 띄어쓰기만으로 형태소 단위 정보를 분리하기 어려움
- 어휘 사전에 없는 새 단어는 미등록 단어(OOV)가 될 수 있음. 드물게 등장하는 단어까지 모두 추가하면 어휘 사전이 커진다

```
review = '별점 구경 평가 가능? cg. 시작과 즐거움을 보고! 여행 ost 는 어디에 최고!!'
tokenized = review.split()
```

- split()은 구분자를 지정하지 않으면 공백을 기준으로 나눈다

→ 문장부호가 단어에 붙은 채 남을 수 있음

예제 5.1 단어 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거음은 최고! 더불어 ost는 더더욱 최고!!"
tokenized = review.split()
print(tokenized)
```

출력 결과

```
['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거음은', '최고!', '더불어', 'ost는', '더더욱',
'최고!!']
```

2. 글자 토큰화(Character Tokenization)

- 글자 토큰화는 텍스트를 글자 단위로 나누는 방법
- 단어보다 단위가 작으므로 미등록 단어 문제를 줄일 수 있고, 희귀한 단어나 새 단어도 글자들의 조합으로 표현할 수 있음

↔ 입력 시퀀스 길이가 길어질 수 있고, 글자 하나만으로는 단어의 의미를 충분히 담기 어려움

```
tokenized = list(review)
```

- 교재 예제 5.2 에서는 list()를 이용해 문자열을 글자별 원소로 변환
- Python 의 문자열을 순회하므로 공백과 구두점 역시 개별 문자로 취급될 수 있음
- 한국어는 한 글자가 자음과 모음으로 구성되므로, 자모 단위로 더 세분하는 방식도 가능하다
- jamo 라이브러리를 사용해 한글을 자모 단위로 분해
- 글자를 더 작게 나누면 어휘 종류는 줄어들지만, 토큰 수는 늘어나며 원래 글자로 복원할 정보가 필요해진다

```
import jamo
from jamo import j2hcj
retval = jamo.h2j(review)
decomposed = j2hcj(retval)
tokenized = list(decomposed)
```

예제 5.2 글자 토큰화

```
review = "현실과 구분 불가능한 cg. 시각적 즐거음은 최고! 더불어 ost는 더더욱 최고!!"
tokenized = list(review)
print(tokenized)
```

출력 결과

```
['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g', '.', ' ', '시',
'각', '적', ' ', '즐', '거', '음', '은', ' ', '최', '고', '!', ' ', '더', '불', '어', ' ', 'o', 's',
't', '는', ' ', '더', '더', '욱', ' ', '최', '고', '!', '!']
```

자모 변환 함수

```
retval = jamo.h2j(
    hangul_string
)
```

한글 호환성 자모 변환 함수

```
retval = jamo.j2hcj(
    jamo
)
```

예제 5.3 자소 단위 토큰화

```
from jamo import h2j, j2hcj

review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
decomposed = j2hcj(h2j(review))
tokenized = list(decomposed)
print(tokenized)
```

출력 결과

```
['ㅎ', 'ㅕ', 'ㄴ', 'ㅅ', 'ㅣ', 'ㄹ', 'ㄱ', 'ㅘ', ' ', 'ㄱ', 'ㅜ', 'ㅂ', 'ㅜ', 'ㄴ', ' ', 'ㅂ', 'ㅜ',
'ㄹ', 'ㄱ', 'ㅏ', 'ㄴ', 'ㅡ', 'ㅇ', 'ㅎ', 'ㅏ', 'ㄴ', ' ', 'c', 'g', '.', ' ', 'ㅅ', 'ㅣ', 'ㄱ',
'ㅏ', 'ㄱ', 'ㅈ', 'ㅓ', 'ㄱ', ' ', 'ㅈ', 'ㅡ', 'ㄹ', 'ㄱ', 'ㅓ', 'ㅇ', 'ㅡ', 'ㅁ', 'ㅇ', 'ㅡ', 'ㄴ',
' ', 'ㅊ', 'ㅚ', 'ㄱ', 'ㅗ', '!', ' ', 'ㄷ', 'ㅓ', 'ㅂ', 'ㅜ', 'ㄹ', 'ㅇ', 'ㅓ', ' ', 'o', 's',
't', 'ㄴ', 'ㅡ', 'ㄴ', ' ', 'ㄷ', 'ㅓ', 'ㄷ', 'ㅓ', 'ㅇ', 'ㅜ', 'ㄱ', ' ', 'ㅊ', 'ㅚ', 'ㄱ', 'ㅗ',
'!', '!']
```

형태소 토큰화

- 형태소 토큰화(Morpheme Tokenization)는 언어에서 의미를 지니는 최소 단위인 형태소를 기준으로 텍스트를 분리
- 한국어는 조사가 붙거나 어미가 변하는 교착어(Agglutinative Language)의 특징이 있어 형태소를 구분하는 접근이 유용
- ex) "나는 학교에 갔다"를 그대로 공백 분리하면 "나는 / 학교에 / 갔다"로 나뉘지만, 형태소 분석은 "나 / 는 / 학교 / 에 / 가 / 았 / 다"처럼 문법적 요소를 더 세분해 볼 수 있음
- 실제 분리 결과는 분석기와 사전에 따라 달라짐
- 자립 형태소: 문장에서 단독으로 사용될 수 있는 형태소(예: 나, 사과)
- 의존 형태소: 단독 사용이 어렵고 다른 형태소에 붙어 쓰이는 형태소(예: 조사, 어미)
- 형태소 분석과 함께 품사 태깅(Part Of Speech, POS)을 수행하면 각 토큰이 명사·동사·조사 등의 어떤 역할을 하는지 확인할 수 있음

1. 형태소 어휘 사전(Morpheme Vocabulary)

- 형태소 어휘 사전은 형태소와 품사 정보 등을 이용해 문장을 분석할 때 참고하는 사전
- 문장 내 단어의 구조를 파악하려면 어휘 사전과 형태소 분석기 등의 정보를 활용할 수 있음
- 같은 문장이라도 사전의 규모와 분석 규칙에 따라 결과가 달라질 수 있음

2. KoNLPy

- KoNLPy 는 한국어 정보 처리를 위한 라이브러리로, 형태소 분석·품사 태깅 등 기능을 제공
- 책에서는 Okt, Kkma 등 분석기를 소개하며 동일한 문장을 어떻게 다르게 분석하는지 비교

sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

```
from konlpy.tag import Okt
okt = Okt()
nouns = okt.nouns(sentence)
phrases = okt.phrases(sentence)
morphs = okt.morphs(sentence)
pos = okt.pos(sentence)
```

- nouns(): 문장에서 명사를 추출
- phrases(): 구(phrase) 단위 표현을 추출
- morphs(): 문장을 형태소 단위로 분리
- pos(): 형태소와 품사 태그를 함께 반환
- Okt 의 결과는 분석 단위에 따라 달라짐 nouns()와 morphs()의 출력 개수 및 내용이 서로 다른 이유는 명사만 추출하는지, 전체 형태소를 추출하는지의 차이 때문

```
from konlpy.tag import Kkma
kkma = Kkma()
nouns = kkma.nouns(sentence)
sentences = kkma.sentences(sentence)
morphs = kkma.morphs(sentence)
pos = kkma.pos(sentence)
```

- Kkma 는 명사 추출, 문장 분리, 형태소 분리, 품사 태깅 등의 메서드를 제공
- 같은 문장에 Okt 와 Kkma 를 적용했을 때 차이가 나타날 수 있으며, 어느 쪽이 항상 정확하다고 단정하기보다 데이터의 특성과 목적에 맞춰 분석기를 선택해야

표 5.1 Okt 품사 태그

태그	품사	태그	품사
Noun	명사	Punctuation	구두점
Verb	동사	Foreign	외국어
Adjective	형용사	Alpha	알파벳
Determiner	관형사	Number	숫자
Adverb	부사	KoreanParticle	자음/모음
Conjunction	접속사	URL	웹 주소
Exclamation	감탄사	Hashtag	해시태그(#)
Josa	조사	Email	이메일
PreEomi	선어말어미	ScreenName	아이디 표기(@)
Eomi	어미	Unknown	미등록
Suffix	접미사		

표 5.2 꼬꼬마 품사 태그

태그	품사	태그	품사
NNG	보통명사	EPH	존칭 선어말 어미
NNP	고유명사	EPT	시제 선어말 어미
NNB	일반 의존 명사	EPP	공손 선어말 어미
NNM	단위 의존 명사	EFN	평서형 종결 어미
NR	수사	EFQ	의문형 종결 어미
NP	대명사	EFO	명령형 종결 어미
VV	동사	EFA	청유형 종결 어미
VA	형용사	EFI	감탄형 종결 어미
VXV	보조 동사	EFR	존칭형 종결 어미
VXA	보조 형용사	ECE	대등 연결 어미
VCP	긍정 지정사	ECS	보조적 연결 어미
VCN	부정 지정사	ECD	의존적 연결 어미
MDN	수 관형사	ETN	명사형 전성 어미
MDT	일반 관형사	ETD	관형형 전성 어미

3. NLTK

- NLTK(Natural Language Toolkit)는 자연어 처리를 위한 Python 라이브러리로, 토큰화와 품사 태깅 등 여러 언어 처리 기능을 제공
- 영어 문장을 대상으로 word_tokenize()와 pos_tag()를 실습

```
import nltk
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
from nltk import word_tokenize, pos_tag
```

```
sentence = "Those who can imagine anything, can create the impossible."
word_tokens = word_tokenize(sentence)
pos = pos_tag(word_tokens)
```

- word_tokenize()는 문장을 단어와 문장부호 등으로 나누고, pos_tag()는 분리된 토큰에 품사 태그를 부착
- 교재의 품사 태그 표는 명사(NN), 동사(VB) 등 품사 구분의 예시를 제공
- NLTK 예제는 영어 중심으로 소개되어 있으므로 한국어 형태소 분석을 그대로 대체하는 접근은 아니다

4. spaCy

- spaCy 는 Python 및 Cython 기반의 자연어 처리 라이브러리
- 토큰화와 품사 분석 외에도 다양한 언어 처리 결과를 Doc 와 Token 객체의 속성으로 확인할 수 있음

```
import spacy
nlp = spacy.load('en_core_web_sm')
doc = nlp(sentence)
for token in doc:
    print(token.pos_, token.tag_, token.text)
```

- token.text: 토큰의 실제 문자열
- token.pos_: 일반적인 품사 범주를 표현
- token.tag_: 더욱 세분화된 품사 태그를 표현
- token.lemma_: 기본형(표제어)에 관한 정보를 제공
- spaCy 의 입력 문장 전체는 Doc 객체로, 각 단어는 Token 객체로 다룸
- 문장과 토큰이 객체로 관리되기 때문에 품사, 원형, 위치 등 부가 정보를 편리하게 확인할 수 있음

예제 5.9 spaCy 품사 태깅

```
import spacy

nlp = spacy.load("en_core_web_sm")
sentence = "Those who can imagine anything, can create the impossible."
doc = nlp(sentence)

for token in doc:
    print(f"[{token.pos_:5} - {token.tag_:3}] : {token.text}")
```

출력 결과

```
[PRON  - DT ] : Those
[PRON  - WP ] : who
[AUX   - MD ] : can
[VERB  - VB ] : imagine
[PRON  - NN ] : anything
[PUNCT - ,  ] : ,
[AUX   - MD ] : can
[VERB  - VB ] : create
[DET   - DT ] : the
[ADJ   - JJ ] : impossible
[PUNCT - .  ] : .
```

하위 단어 토큰화

- 하위 단어 토큰화(Subword Tokenization)는 단어를 통째로 사용하는 방식과 글자 단위로 나누는 방식 사이의 절충 방법
- 자주 등장하는 단어 또는 문자열은 하나의 토큰으로 유지하고, 희귀한 단어는 여러 하위 단위로 분해할 수 있음
- 단어 토큰화에서 발생하는 미등록 단어 문제를 완화하며, 단어 전체를 어휘 사전에 등록하는 것보다 효율적으로 어휘 사전 크기를 관리할 수 있음
- 하위 단어 방식으로 바이트 페어 인코딩, 센텐스피스, 워드피스를 차례대로 설명

1. 바이트 페어 인코딩(Byte Pair Encoding, BPE)

- BPE 는 원래 데이터 압축에서 사용된 방법이며, 자연어 처리에서는 말뭉치에서 함께 자주 등장하는 문자 또는 기호 쌍을 반복적으로 병합해 하위 단어 어휘를 만든다
  - 초기 단위로 글자 등의 작은 단위를 설정하고 각 단어를 그 단위로 분할
  - 현재 단위들 중 서로 인접하며 자주 등장하는 쌍의 빈도를 계산
  - 가장 빈도가 높은 쌍을 하나의 새 토큰으로 병합
  - 설정한 병합 횟수 또는 어휘 크기에 도달할 때까지 반복

- 반복적으로 함께 나타나는 조각이 점차 하나의 토큰으로 만들어지므로, 흔한 문자열은 짧은 토큰 시퀀스로 표현되고 드문 문자열은 여러 토큰으로 남게 됨
- 책의 예시는 문자열의 인접 쌍 빈도를 세고 병합 결과를 갱신하는 과정을 단계별로 제시

2. 센텐스피스(SentencePiece)

- 센텐스피스는 텍스트를 하위 단어 단위로 나누는 토크나이저를 구축하고 사용할 수 있도록 하는 도구
- 교재에서는 SentencePiece 와 한국어 말뭉치 KorPora 를 이용하여 토크나이저를 학습하고 인코딩·디코딩하는 흐름을 실습

```
from Korpora import Korpora
corpus = Korpora.load('korean_petitions')
dataset = corpus.train
```

- 먼저 말뭉치에서 문장을 수집해 텍스트 파일로 저장
- 파일이 토크나이저 학습의 입력으로 사용됨
- 한국어 청원 데이터셋을 예제로 사용

```
from sentencepiece import SentencePieceTrainer
SentencePieceTrainer.Train('--input=./datasets/corpus.txt --model_prefix=petition_bpe --vocab_size=8000 --model_type=bpe')
```

- input: 학습에 사용할 말뭉치 파일 경로
- model_prefix: 학습한 토크나이저 모델 및 어휘 파일의 접두어
- vocab_size: 어휘 사전의 크기
- model_type: BPE, unigram 등 사용할 하위 단어 알고리즘 유형

```
from sentencepiece import SentencePieceProcessor
tokenizer = SentencePieceProcessor()
tokenizer.Load('petition_bpe.model')
tokenized_sentence = tokenizer.encode_as_pieces(sentence)
encoded_sentence = tokenizer.encode_as_ids(sentence)
decoded_sentence = tokenizer.decode_ids(encoded_sentence)
```

- encode_as_pieces()는 토큰 문자열을, encode_as_ids()는 어휘 사전에 대응한 정수 ID 를 돌려준다. decode_ids()와 decode_pieces()는 인코딩된 데이터를 다시 문자열로 복원할 때 사용
- 이 메서드들의 출력을 비교
- 센텐스피스의 출력에는 단어 경계 또는 공백 정보를 표현하는 특수 기호가 포함될 수 있음
- 공백을 단순히 삭제하지 않고 토큰화와 복원 과정에서 다루는 점이 특징

예제 5.13 바이트 페어 인코딩 토큰화

```
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")

sentence = "안녕하세요, 토크나이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토크나이저를 사용할 수 있답니다"]

tokenized_sentence = tokenizer.encode_as_pieces(sentence)
tokenized_sentences = tokenizer.encode_as_pieces(sentences)
```

```
print("단일 문장 토큰화 :", tokenized_sentence)
print("여러 문장 토큰화 :", tokenized_sentences)

encoded_sentence = tokenizer.encode_as_ids(sentence)
encoded_sentences = tokenizer.encode_as_ids(sentences)
print("단일 문장 정수 인코딩 :", encoded_sentence)
print("여러 문장 정수 인코딩 :", encoded_sentences)

decode_ids = tokenizer.decode_ids(encoded_sentences)
decode_pieces = tokenizer.decode_pieces(encoded_sentences)
print("정수 인코딩에서 문장 변환 :", decode_ids)
print("하위 단어 토큰에서 문장 변환 :", decode_pieces)
```

출력 결과

```
단일 문장 토큰화 : ['_안녕하세요', ',', '_토', '크', '나', '이', '저', '가', '_잘', '_학', '습',
'되었', '군요', '!']
여러 문장 토큰화 : [['_이렇게', '_입', '력', '값을', '_리', '스트', '로', '_받아서'], ['_쉽게',
'_토', '크', '나', '이', '저', '를', '_사용할', '_수', '_있', '답니다']]
단일 문장 정수 인코딩 : [664, 6553, 991, 6880, 6544, 6513, 6590, 6523, 159, 110, 6554, 868, 782,
6648]
여러 문장 정수 인코딩 : [[370, 180, 6677, 4427, 1768, 1610, 6527, 4157], [1677, 991, 6880, 6544,
6513, 6590, 6536, 5848, 18, 5, 2633]]
정수 인코딩에서 문장 변환 : ['이렇게 입력값을 리스트로 받아서', '쉽게 토크나이저를 사용할 수
있답니다']
하위 단어 토큰에서 문장 변환 : ['이렇게 입력값을 리스트로 받아서', '쉽게 토크나이저를 사용할 수
있답니다']
```

3. 워드피스(WordPiece)

- 워드피스는 하위 단어 토큰화 방식으로, 단어를 통째로 어휘 사전에 등록하지 않고 학습된 하위 토큰의 조합으로 분리
- 워드피스의 토큰 병합 판단에 사용되는 점수와 Hugging Face tokenizers 라이브러리의 학습·추론 방법을 소개

- 교재의 점수는 함께 등장하는 쌍의 빈도만 보는 것이 아니라 각 개별 토큰의 빈도까지 고려하는 식으로 제시됨

→ 단순히 가장 많이 등장하는 쌍을 병합하는 BPE 와 학습 기준을 구분해야

점수의 형태: score = freq(x,y) / (freq(x) × freq(y))

```
from tokenizers import Tokenizer
from tokenizers.models import WordPiece
from tokenizers.normalizers import Sequence, NFD, Lowercase
from tokenizers.pre_tokenizers import Whitespace
tokenizer = Tokenizer(WordPiece())
tokenizer.normalizer = Sequence([NFD(), Lowercase()])
tokenizer.pre_tokenizer = Whitespace()
```

- 정규화(Normalization)는 텍스트를 통일된 형식으로 바꾸는 절차
- 사전 토큰화(Pre-tokenization)는 본격적인 하위 단어 분할 전 문장을 일정한 초기 단위로 구분
- 토크나이저 학습 후에는 인코딩 결과에 포함된 tokens 와 ids 를 확인하고 디코더를 지정해 복원할 수 있음

```
from tokenizers.decoders import WordPiece as WordPieceDecoder
tokenizer.decoder = WordPieceDecoder()
```

- 교재의 워드피스 실습은 토크나이저 구성 → 정규화 및 사전 토큰화 → 학습 → 인코딩 → 토큰 ID 확인 → 디코딩 순으로 진행됨

예제 5.16 워드피스 토큰화

```
from tokenizers import Tokenizer
from tokenizers.decoders import WordPiece as WordPieceDecoder

tokenizer = Tokenizer.from_file("../models/petition_wordpiece.json")
tokenizer.decoder = WordPieceDecoder()
```

```
sentence = "안녕하세요, 토크나이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토크나이저를 사용할 수 있답니다"]

encoded_sentence = tokenizer.encode(sentence)
encoded_sentences = tokenizer.encode_batch(sentences)

print("인코더 형식 :", type(encoded_sentence))

print("단일 문장 토큰화 :", encoded_sentence.tokens)
print("여러 문장 토큰화 :", [enc.tokens for enc in encoded_sentences])

print("단일 문장 정수 인코딩 :", encoded_sentence.ids)
print("여러 문장 정수 인코딩 :", [enc.ids for enc in encoded_sentences])

print("정수 인코딩에서 문장 변환 :", tokenizer.decode(encoded_sentence.ids))
```

출력 결과

```
인코더 형식 : <class 'tokenizers.Encoding'>
단일 문장 토큰화 : ['안녕하세요', '.', '워', '##드', '##피', '##스', '토', '##크', '##나이', '##저',
'##입니다', '.']
여러 문장 토큰화 : [['이렇', '##게', '입력', '##값을', '리스트', '##로', '받아서'], ['쉽게', '토', '##크',
'##나이', '##저', '##를', '사용할', '수', '있습니다', '.']]
단일 문장 정수 인코딩 : [8760, 13, 10398, 7638, 7955, 7512, 8693, 8415, 16269, 7536, 7598, 13]
여러 문장 정수 인코딩 : [[19643, 13834, 28119, 7495, 11589, 7502], [9739, 8693, 8415, 16269, 7536,
7510, 14129, 7562, 7698]]
정수 인코딩에서 문장 변환 : 안녕하세요. 워드피스 토크나이저입니다.
```

6. 임베딩

- 임베딩(Embedding)은 텍스트의 단어를 컴퓨터가 학습에 활용할 수 있도록 수치 벡터 형태로 나타내는 방법
- 교재 6 장은 원-핫 인코딩을 시작으로 언어 모델, N-gram, TF-IDF, Word2Vec 을 차례대로 소개

원-핫 인코딩(One-Hot Encoding)

- 원-핫 인코딩은 어휘 사전에 포함된 단어를 벡터로 나타내되, 특정 단어에 해당하는 위치의 원소만 1 이고 나머지는 모두 0 인 벡터로 표현
- ex) 어휘 사전이 [사과, 바나나, 포도, 수박]이면 "사과"는 [1,0,0,0], "바나나"는 [0,1,0,0], "포도"는 [0,0,1,0]으로 표현할 수 있음
- 이 예시는 원리를 설명하기 위한 예시이며 교재 표의 실제 어휘와는 별개
- 어휘 사전 크기가 N 이면 원-핫 벡터의 차원도 N
- 각 단어가 하나의 고유한 위치를 가짐으로써 구분은 가능하지만, 단어 사이 의미적 관계를 벡터 값만으로 표현하기 어려움
- 대부분이 0 이므로 희소 벡터(Sparse Vector)이며, 어휘 종류가 많아질수록 차원이 커진다

표 6.1 원-핫 인코딩 매핑 테이블

색인	0	1	2	3
토큰	I	like	apples	bananas

언어 모델(Language Model)

- 언어 모델은 입력된 문장으로 다음에 등장할 문장이나 단어의 확률을 계산하는 모델
- 확률을 활용해 자동 번역, 음성 인식, 텍스트 요약 등 여러 자연어 처리 작업을 수행할 수 있음
- 문장 전체의 가능한 조합에 확률을 직접 부여하기는 어렵기 때문에, 단어의 등장 순서를 고려해 조건부 확률로 다음 단어를 예측하는 방향을 취

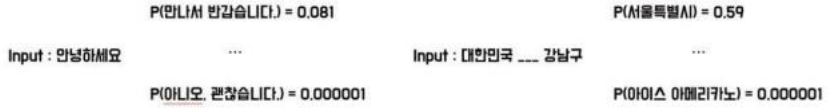


그림 6.1 언어 모델

자기회귀 언어 모델(Autoregressive Language Model)

- 자기회귀 언어 모델은 입력된 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측
- 앞서 나온 토큰들의 시퀀스가 주어지면 다음 토큰의 확률을 계산하고, 그 결과를 다시 다음 예측에 활용하는 과정을 반복

  조건부 확률: $P(w_n \mid w_1, w_2, \ldots, w_{n-1}) = P(w_1, \ldots, w_n) / P(w_1, \ldots, w_{n-1})$
- 주어진 문장에서 다음에 등장할 단어의 확률을 매번 계산하는 것은 많은 경우의 수를 요구
- 그래서 다음 단어를 예측할 때 앞의 모든 단어를 정확히 다루는 대신, 일정한 개수의 이전 단어만 참고하는 N-gram 접근이 소개됨

N-gram

- N-gram 은 연속된 N 개의 단어를 하나의 묶음으로 다루어 앞의 N−1 개 단어를 기반으로 다음 단어의 등장 가능성을 추정하는 방법
- Unigram(1-gram): 단어 하나를 하나의 단위로 본다
- Bigram(2-gram): 연속된 단어 두 개를 하나의 단위로 본다
- Trigram(3-gram): 연속된 단어 세 개를 하나의 단위로 본다
- ex) "I like this movie"라는 문장에서 unigram 은 [I], [like], [this], [movie], bigram 은 [I like], [like this], [this movie], trigram 은 [I like this], [like this movie]로 구성됨
- (방법 이해를 위한 예시)
- 일반적으로 N-gram 모델에서는 지금까지의 모든 단어를 이용하지 않고 최근 N−1 개 단어만 참고하는 마르코프 가정에 기반해 확률을 추정
- ex) bigram 이라면 $P(w_n \mid w_{n-1})$, trigram 이라면 $P(w_n \mid w_{n-2}, w_{n-1})$의 형태를 사용
- 단어가 등장하는 횟수로 조건부 확률을 근사하며, 특정 조합이 학습 데이터에 나타나지 않으면 확률을 계산하기 어렵거나 0 이 될 수 있음
- N 을 늘리면 긴 문맥을 고려할 수 있으나 조합 수가 많아져 데이터 부족 문제가 커질 수 있음



그림 6.3 N이 1, 2, 3인 N-gram

TF-IDF

TF-IDF(Term Frequency-Inverse Document Frequency)는 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법으로, 문서 내 단어의 중요도를 평가하는 데 사용되는 통계적인 가중치를 의미한다. BoW(Bag-of-Words)에 가중치를 부여하는 방식으로 볼 수 있다.

- BoW 는 텍스트에 나타나는 단어의 등장 여부나 빈도를 표현
- 문장 내 단어의 순서보다는 어떤 단어가 얼마나 등장했는지에 주목

→ 같은 단어로 구성되면 문장의 순서가 달라도 표현이 같아질 수 있음

문서	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

1. 단어 빈도(Term Frequency, TF)

- 단어 빈도(TF)는 하나의 문서에서 특정 단어가 얼마나 자주 나타나는지를 의미
- 단어가 문서 안에 많이 등장할수록 중요하다고 볼 수 있지만, "the", "is"와 같이 여러 문서에서 흔히 나타나는 단어까지 높은 점수를 줄 수 있다는 한계가 있음

TF(t,d) = 문서 d 에서 단어 t 가 등장한 횟수

2. 문서 빈도(Document Frequency, DF)

- 문서 빈도(DF)는 단어가 전체 문서 중 몇 개 문서에 포함되는지를 표현
- 한 문서에서 단어가 여러 번 등장하더라도 DF 는 문서를 하나로 센다
- 모든 문서에 등장하는 흔한 단어의 DF 는 높음

DF(t,D) = 문서 집합 D 에서 단어 t 가 등장한 문서의 수

3. 역문서 빈도(Inverse Document Frequency, IDF)

- 역문서 빈도(IDF)는 많은 문서에서 흔하게 등장하는 단어의 가중치를 낮추고, 비교적 적은 문서에서 등장하는 단어의 가중치를 높이기 위한 값
- DF 값의 역수에 로그를 적용하는 방식을 제시

  IDF(t,D) = log(count(D) / (1 + DF(t,D)))
- 교재 수식에서는 분모에 1 을 더해 DF 가 0 일 때의 계산 문제를 방지
- 또한 문서 수와 빈도의 비율에 로그를 취해 수치의 크기를 조정
- 이 식은 교재에 나온 정의를 옮긴 것으로, 라이브러리 구현은 서로 다른 평활화 방식을 사용할 수 있음

4. TF-IDF

- TF-IDF 는 특정 문서에서 단어가 자주 등장하면서도 다른 문서에서는 상대적으로 드물게 등장할 때 큰 가중치를 부여하려는 방법

  TF-IDF(t,d,D) = TF(t,d) × IDF(t,D)

→ 문서에서 여러 번 등장하는 단어가 무조건 중요해지는 것은 아니다

- 문서 집합 전체에서 흔한 단어는 IDF 에 의해 가중치가 줄어들 수 있음

↔ 문서에 특징적인 단어는 상대적으로 높은 값을 획득

수식 6.7 역문서 빈도

$$IDF(t,D) = \log\left(\frac{count(D)}{1+DF(t,D)}\right)$$

수식 6.8 TF-IDF

$$TF-IDF(t,d,D) = TF(t,d) \times IDF(t,d)$$

TF-IDF 모델 실습

- Scikit-learn 라이브러리를 활용하여 TF-IDF 를 계산
- 모델의 학습에 해당하는 fit()은 문서 집합의 단어 사전과 통계 정보를 구축하고, transform()은 이를 이용해 문서를 벡터로 변환. fit_transform()을 이용해 학습과 변환을 연속으로 수행할 수도 있음

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    'This movie is famous movie',
    'I like this movie',
    "I don't like this movie"
]

tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(corpus)
```

```
print(tfidf_matrix.toarray())
print(tfidf_vectorizer.vocabulary_)
```

- TfidfVectorizer()는 문서의 단어 빈도와 역문서 빈도 정보를 활용해 TF-IDF 표현을 만든다. toarray()는 희소 행렬을 일반 배열로 변환하며, 결과의 형태는 (문서 수) × (어휘 사전의 단어 수)
- vocabulary_ 속성은 단어와 단어에 부여된 열 인덱스의 매핑
- 높은 가중치를 받은 단어를 확인하면 문서를 특징짓는 단어가 무엇인지 살펴볼 수 있음
- 값을 기준으로 중요한 단어를 추출하는 예제를 제시
- 다만 TF-IDF 는 등장 빈도에 기반하므로 어순과 문맥을 직접 고려하지 못함
- 또한 단어 자체의 의미가 유사한지까지 벡터에 표현하지는 않는다

예제 6.2 TF–IDF 계산

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "That movie is famous movie",
    "I like that actor",
    "I don't like that actor"
]

tfidf_vectorizer = TfidfVectorizer()
tfidf_vectorizer.fit(corpus)
tfidf_matrix = tfidf_vectorizer.transform(corpus)

print(tfidf_matrix.toarray())
print(tfidf_vectorizer.vocabulary_)
```

출력 결과

```
[[0.         0.         0.39687454 0.39687454 0.         0.79374908 0.2344005 ]
 [0.61980538 0.         0.         0.         0.61980538 0.         0.48133417]
 [0.4804584  0.63174505 0.         0.         0.4804584  0.         0.37311881]]
{'that': 6, 'movie': 5, 'is': 3, 'famous': 2, 'like': 4, 'actor': 0, 'don': 1}
```